

Bushwhacking towards Event-Driven Chorex

ASHTON WIERSDORF, University of Utah, USA

BEN GREENMAN, University of Utah, USA

CCS Concepts: .

Additional Key Words and Phrases: choreographic programming

ACM Reference Format:

Ashton Wiersdorf and Ben Greenman. 2026. Bushwhacking towards Event-Driven Chorex. *Proc. ACM Program. Lang.* 1, 1, Article (May 2026), 3 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Extended Abstract

Every distributed system should be grounded in a choreography that specifies how system components interact and projects down to skeleton code for each component. Choreographies prevent deadlocks and communication errors by design, and they document the global structure of the system in a readable format that has a firm connection to runtime behavior. Choreographies can also support formal verification, and the rapid progress of recent work on the semantics of choreographies [Hirsch and Garg 2022; Samuelson, Hirsch, and Cecchetti 2025; Graversen, Hirsch, and Montesi 2024; Plyukhin, Peressotti, and Montesi 2024; Acclavio, Manara, and Montesi 2025] suggests that the time is ripe to implement full-featured languages that put choreographies in the hands of working programmers.

We are striving to make choreographies an everyday tool for the Elixir community through Chorex, a choreographic language written in Elixir and designed for Elixir programmers. Elixir is a compelling target for several reasons: it compiles to the Beam VM and thus has access to primitives that support low-latency, distributed, fault-tolerant systems; it comes with a modern package system and metaprogramming system, which allows Chorex to integrate as smoothly as a regular library [Tobin-Hochstadt et al. 2011]; and it has a sizeable userbase. For example, the Phoenix web framework is written in Elixir and has over 55,000 users according to GitHub [GitHub, n.d.] and was ranked the most loved web framework on the 2025 StackOverflow developer survey [StackOverflow 2025]. Chorex [Wiersdorf and Greenman 2025] can express choreographies from the literature and the core logic of Elixir applications. We have built several demos including a higher-order bookseller [Hirsch and Garg 2022] and a TCP socket server.

There are, however, significant barriers standing between Chorex and applications such as the Phoenix web framework. Consider a very simple database server in which a Client logs in, gets assigned to a Worker process, and issues requests to the Worker. Suppose the Client asks the Worker to perform several expensive database queries and then disconnects before the queries finish:

- The Worker should receive an out-of-band notification that the connection closed and should stop processing to save resources—but to our knowledge, this protocol does not fit into any existing choreographic language.

Authors' Contact Information: Ashton Wiersdorf, University of Utah, Salt Lake City, Utah, USA, research@wiersdorfmail.net; Ben Greenman, University of Utah, Salt Lake City, Utah, USA, benjaminlgreenman@gmail.com.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

© 2026 Copyright held by the owner/author(s).

ACM 2475-1421/2026/5-ART

<https://doi.org/XXXXXXX.XXXXXXX>

- There might be several (Client, Worker) pairs running in parallel, in which case the choreography should know to terminate only the troublesome pair.
- Any reasonable server should be able to dynamically resize its Worker pool depending on the number of Clients who connect, but on-the-fly process creation is a challenge.

These difficulties stem from the fact that traditional choreographies are organized around the notion of a straight-line session between a fixed set of participants. A broader view is needed for choreographies to be able to express process lifecycles.

We are building *Event-Driven Chorex*, a language that significantly expands the expressiveness of baseline Chorex through the following features:

- **Events** as reactive functions that are triggered by external happenings and redirect the flow of a choreography.
- **Census primitives** to remove the restriction that choreographies work with a fixed number of processes and instead allow them to create, group, and destroy processes as-needed.
- **First-class process references** allow for dynamically creating and involving processes as part of (sub-)choreographies.

Event-driven Chorex decouples processes from the roles they play in a communication, and enriches the baseline Chorex language with a process-oriented API. In terms of our running example, an Event-driven choreography would begin by declaring the relationships between the Client and Worker roles and would include an event handler that terminates a specific Client's Worker (`@self.worker`) when the Client disconnects:

```
role Client,
  worker: Worker

role Worker,
  client: Client

event Client, {:error, :closed} do
  suspend([@self, @self.worker], @self.@state[:client_id])
end
```

Getting such code to run requires a completely new compiler, as the original implementation of Chorex was designed to support a fixed set of participants and left process creation as the job of the caller of the choreography, external to the Chorex DSL. In Event-Driven Chorex, the target language is roughly the same as before (Elixir GenServers), but the compiler back-end is much more sophisticated and the source language is expanded. We remark that our focus is on building an expressive implementation rather than on formal verification. Event-Driven Chorex eventually needs a strong metatheory, but this will come after we discover what must be done to express Elixir programs with modest effort.

In this talk, we will present the motivation for Event-Driven Chorex and report the current work-in-progress of the language implementation. We have so far implemented a role-based typechecker and we can compile a “hello world” program consisting of a single send and receive. This first example consists of approximately 6 lines of source code and compiles to over 150 lines total between the two participants and their necessary glue code. There is also a substantial standard library of Chorex communication utilities. Over the next few months, we plan to support the database example sketched above and a basic pub-sub architecture where subscribers can leave anytime. The talk will present a variety of example choreographies to relate our goals to concrete code, and to facilitate discussion with the audience. One key point that we hope to discuss is exactly *when* events should be allowed to interrupt a choreography, as arbitrary interruption via arbitrary events could lead to unintuitive programming, misconfigured state rollbacks, or poor performance.

Bibliography

- Matteo Acclavio, Giulia Manara, and Fabrizio Montesi. 2025. Formulas as Processes, Deadlock-Freedom as Choreographies. In *ESOP*. Springer, 23–55. https://doi.org/10.1007/978-3-031-91118-7_2
- GitHub. n.d. Dependency Graph: phoenix-framework/phoenix. Accessed March 12, 2026. <https://github.com/phoenixframework/phoenix/network/dependents>
- Eva Graversen, Andrew K. Hirsch, and Fabrizio Montesi. 2024. Alice or Bob?: Process Polymorphism in Choreographies. *Journal of Functional Programming* 34. <https://doi.org/10.1017/S0956796823000114>
- Andrew K. Hirsch and Deepak Garg. 2022. Pirouette: Higher-Order Typed Functional Choreographies. *Proceedings of the ACM on Programming Languages* 6, POPL (2022), 1–27. <https://doi.org/10.1145/3498684>
- Dan Plyukhin, Marco Peressotti, and Fabrizio Montesi. 2024. Ozone: Fully Out-of-Order Choreographies. In *ECOOP*. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 31:1–28. <https://doi.org/10.4230/LIPICS.ECOOP.2024.31>
- Ashley Samuelson, Andrew K. Hirsch, and Ethan Cecchetti. 2025. Choreographic Quick Changes: First-Class Location (Set) Polymorphism. *Proceedings of the ACM on Programming Languages* 9, OOPSLA2 (2025), 1783–1808. <https://doi.org/10.1145/3763114>
- StackOverflow. 2025. 2025 Developer Survey: Web Frameworks and Technologies. 2025. <https://survey.stackoverflow.co/2025/technology/#2-web-frameworks-and-technologies>
- Sam Tobin-Hochstadt, Vincent St-Amour, Ryan Culpepper, Matthew Flatt, and Matthias Felleisen. 2011. Languages as Libraries. In *PLDI*, 132–41. <https://doi.org/10.1145/1993498.1993514>
- Ashton Wiersdorf and Ben Greenman. 2025. Chorex: Restartable, Language-Integrated Choreographies. *Programming* 10, 3 (2025), 20:1–30. <https://doi.org/10.22152/programming-journal.org/2025/10/20>